

## Module 3 Activity 2: Using the Search Cursor

### Purpose:

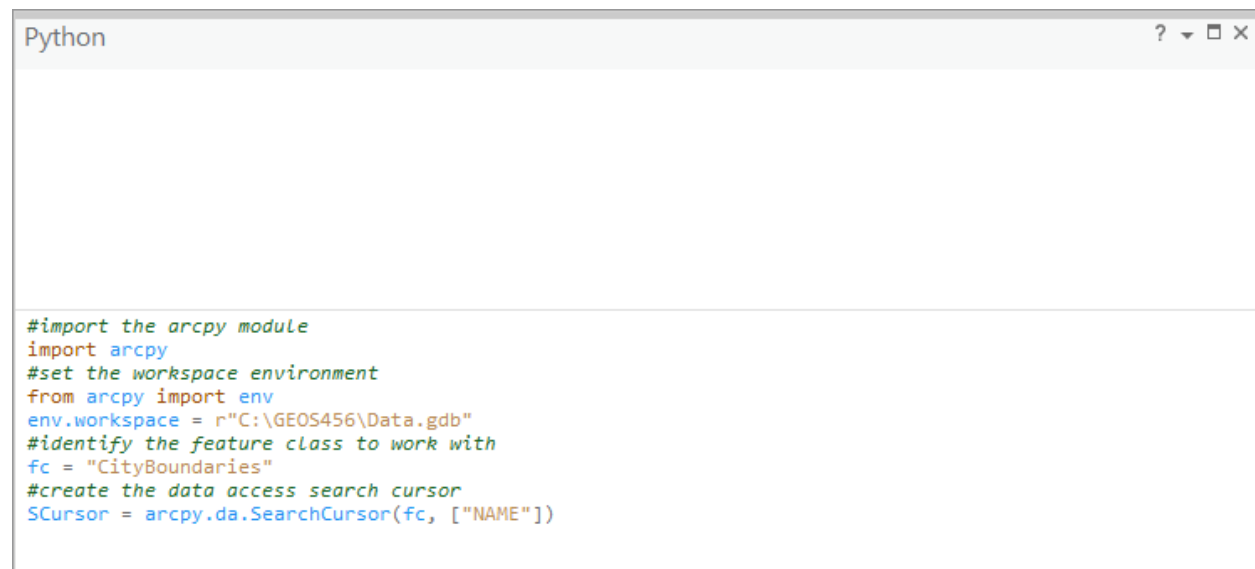
Use the ***SearchCursor()*** function to iterate through a list of fields in a feature class

### Instructions:

You can use any Python editor for this exercise.

#### Part 1:

1. Import the ***arcpy*** module
2. Set the workspace
3. Identify which feature class you want to work with
4. Create the ***SearchCursor*** inside a variable called ***SCursor***
5. Reference the feature class in the ***SearchCursor*** syntax and specify the field you want to search. Fields are a required parameter in the ***SearchCursor*** and are referenced using square brackets ***[]***



```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#identify the feature class to work with
fc = "CityBoundaries"
#create the data access search cursor
SCursor = arcpy.da.SearchCursor(fc, ["NAME"])
```

6. Start a ***for*** loop with the condition to iterate through the records
7. Use a print statement that will return the values of a specified field with string formatting to organize your results

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#specify the feature class to work with
fc = "CityBoundaries"
#create the data access search cursor
SCursor = arcpy.da.SearchCursor(fc, ["NAME"])
#start a for loop and return results of the cursor
for row in SCursor:
    #use string formatting to organize your results
    print ("Name: " + row[0])    #note the index value needs to reference the field value location
```

### NOTES:

- Using the “row” variable in the *for* loop is generally what is used. Like any variable, you can choose which ever name you want.

The index values are referencing the location of the name in the field as well as the location of the cursor. In this case, only one field is being referenced, therefore the row is set to **0**. Remember that all indexing in Python starts at **0**.

8. Run your script
9. Examine your results

```
Python
Name: HELOTES
Name: FAIR OAKS RANCH
Name: GARDEN RIDGE
Name: LYTLE
Name: FAIR OAKS RANCH
Name: GARDEN RIDGE
Name: LIVE OAK
Name: SAINT HEDWIG
Name: KIRBY
Name: CONVERSE
Name: SAN ANTONIO
Name: CHINA GROVE
Name: FAIR OAKS RANCH
Name: OLMOS PARK
Name: BALCONES HEIGHTS
Name: GARDEN RIDGE
Name: BULVERDE
Name: HILL COUNTRY VILLAGE
Name: UNIVERSAL CITY
Name: SAINT HEDWIG
```

10. Modify the line of code from **step 5** to include two additional fields **STATUS**, **COUNTY**
11. Use the formatting from **step 7** to return an organized and clean result
12. Run the script

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#specify the feature class to work with
fc = "CityBoundaries"
#create the data access search cursor
SCursor = arcpy.da.SearchCursor(fc, ["NAME", "STATUS", "COUNTY"])
#start a for loop and return results of the cursor
for row in SCursor:
    #use string formatting to organize your results
    print ("Name: " + row[0])    #note the index value needs to reference the field value location
    #the STATUS is at an index location of 1
    print ("Status: " + row[1])
    #note the index value needs to reference the field value location
    #the COUNTY is at an index location of 2
    print ("County: " + row[2])
    #this next empty print statement will add a space between the
    #results for easier reading
    print ("")
```

13. Examine your results

```
Python
Name: HELOTES
Status: HELOTES
County: Bexar

Name: FAIR OAKS RANCH
Status: FAIR OAKS RANCH
County: Bexar

Name: GARDEN RIDGE
Status: GARDEN RIDGE
County: Comal

Name: LYTLE
Status: LYTLE
County: Atascosa

Name: FAIR OAKS RANCH
Status: FAIR OAKS RANCH
County: Comal
```

14. Add more fields, or modify the ones above and practice the formatting technique
15. Save your script and continue to **Part 2** below

## Part 2:

Use a **with** statement together with a **for** loop with the **SearchCursor()**. Using a **with** statement can be advantageous as it will guarantee closure of the loop and automatically release any locks created by cursors regardless if it is successful or not.

1. Use a **with** statement to call the **SearchCursor**
2. The remainder of the code remains the same as **Part 1**

```
Python
#import the arcpy module
import arcpy
#set the workspace environment
from arcpy import env
env.workspace = r"C:\GEOS456\Data.gdb"
#specify the feature class to work with
fc = "CityBoundaries"
#create the data access search cursor using a "with" statement
with arcpy.da.SearchCursor(fc, ["NAME", "STATUS", "COUNTY"]) as SCursor:
#start a for loop and return results of the cursor
for row in SCursor:
    #use string formatting to organize your results
    print ("Name: " + row[0])    #note the index value needs to reference the field value location
    #the STATUS is at an index location of 1
    print ("Status: " + row[1])
    #note the index value needs to reference the field value location
    #the COUNTY is at an index location of 2
    print ("County: " + row[2])
    #this next empty print statement will add a space between the
    #results for easier reading
    print ("")
```

3. Run the script
4. Examine your results